Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Lenovo

C:\Users\Lenovo\OneDrive\Desktop\Python\spyder\L1,L2_Reg.py

2.py | spider03.py | mlr_sklearn_statsmodel.py | Poly_regression.py | svr.py | flask01.py | L1,L2_Reg.py*

```python
import pandas as pd
import numpy as np
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.impute import SimpleImputer

# Load dataset
dataset = pd.read_csv(r"C:\Users\Lenovo\OneDrive\Desktop\Naresh i Note\Aug\22nd,23rd-08\22nd

# Data cleaning
dataset = dataset.drop(['car_name'], axis=1)
dataset['origin'] = dataset['origin'].replace({1: 'america', 2: 'europe', 3: 'asia'})
dataset = pd.get_dummies(dataset, columns=['origin'])
dataset = dataset.replace('?', np.nan)

# Convert all columns to numeric (important!)
dataset = dataset.apply(pd.to_numeric)

# Features & target
X = dataset.drop(['mpg'], axis=1)
y = dataset['mpg']

# Impute missing values (replace NaN with median)
imputer = SimpleImputer(strategy='median')
X = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)

# Scale features
X_s = preprocessing.scale(X)
X_s = pd.DataFrame(X_s, columns=X.columns)

# Train/test split
x_train, x_test, y_train, y_test = train_test_split(X_s, y, test_size=0.30, random_state=1)

# Fit Linear Regression model
regression_model = LinearRegression()
regression_model.fit(x_train, y_train)

# Print coefficients
```

Source Console Object

Usage

Here you can get help of any object by pressing **Ctrl+I** in front of it, either on the Editor or the Console.

Help can also be shown automatically after writing a left parenthesis next to an object. You can activate this behavior in *Preferences* > *Help*.

Help | Variable Explorer | Plots | Files

Console 1/A

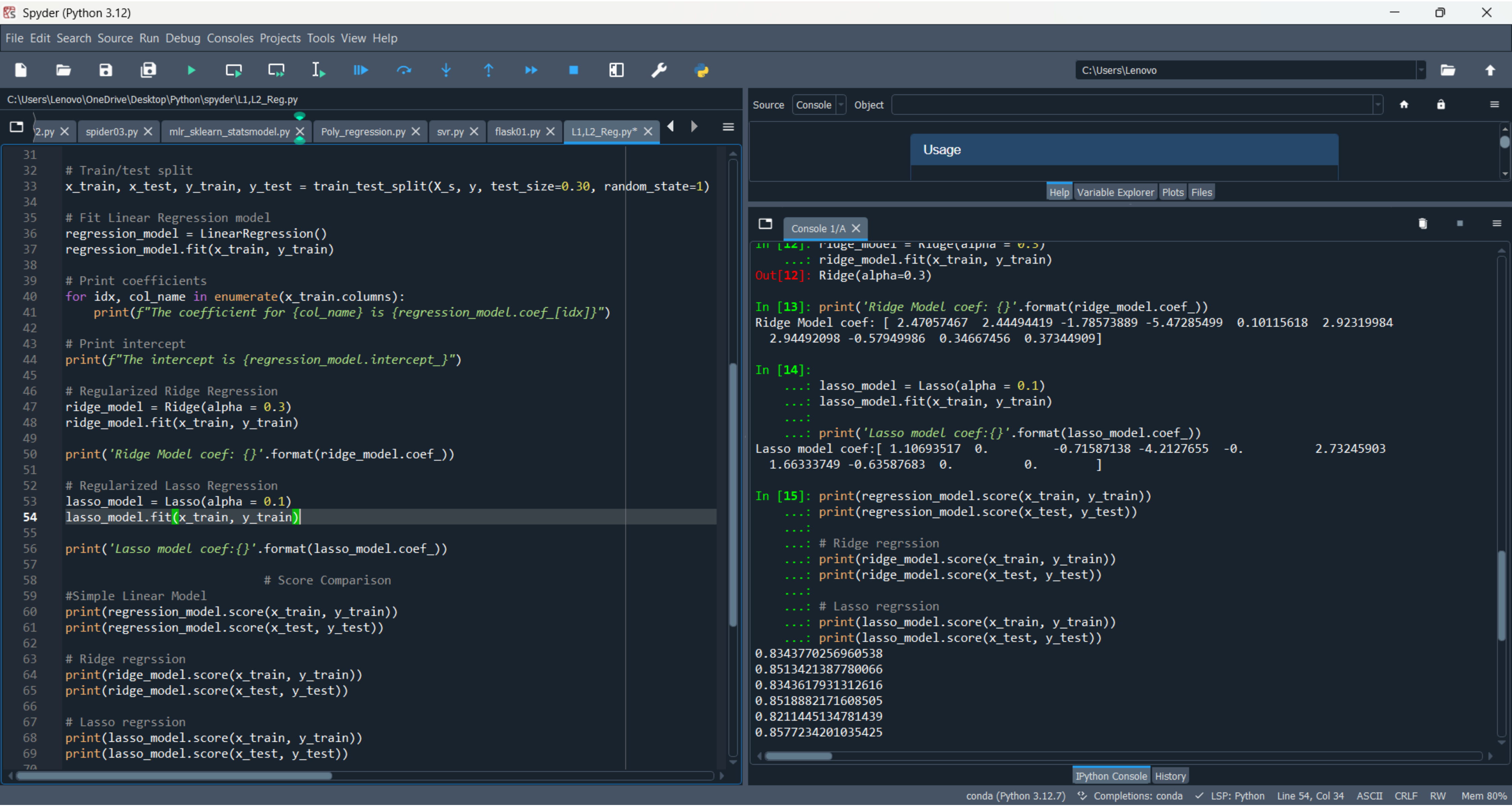
```
In [6]: x_train, x_test, y_train, y_test = train_test_split(X_s, y, test_size=0.30, random_state=1)
   ...:
   ...: # Fit Linear Regression model
   ...: regression_model = LinearRegression()
   ...: regression_model.fit(x_train, y_train)
Out[6]: LinearRegression()

In [7]: for idx, col_name in enumerate(x_train.columns):
   ...:     print(f"The coefficient for {col_name} is {regression_model.coef_[idx]}")
   ...:
   ...: # Print intercept
   ...: print(f"The intercept is {regression_model.intercept_}")
The coefficient for cyl is 2.505951804938502
The coefficient for disp is 2.5357082860560536
The coefficient for hp is -1.7889335736325211
The coefficient for wt is -5.551819873098728
The coefficient for acc is 0.11485734803440915
The coefficient for yr is 2.9318465482116123
The coefficient for car_type is 2.9778697376019396
The coefficient for origin_america is -0.5832955290165992
The coefficient for origin_asia is 0.3474931380432224
The coefficient for origin_europe is 0.3774164680868838
The intercept is 23.665107741982705

In [8]: ridge_model = Ridge(alpha = 0.3)
   ...: ridge_model.fit(x_train, y_train)
```

IPython Console | History

conda (Python 3.12.7) Completions: conda LSP: Python Line 60, Col 1 ASCII CRLF RW Mem 77%


Spyder (Python 3.12)
File Edit Search Source Run Debug Consoles Projects Tools View Help
C:\Users\Lenovo
C:\Users\Lenovo\OneDrive\Desktop\Python\spyder\L1,L2_Reg.py
2.py spider03.py mlr_sklearn_statsmodel.py Poly_regression.py svr.py flask01.py L1,L2_Reg.py*
Train/test split
x_train, x_test, y_train, y_test = train_test_split(X_s, y, test_size=0.30, random_state=1)
# Fit Linear Regression model
regression_model = LinearRegression()
regression_model.fit(x_train, y_train)
# Print coefficients
for idx, col_name in enumerate(x_train.columns):
print(f"The coefficient for {col_name} is {regression_model.coef_[idx]}")
# Print intercept
print(f"The intercept is {regression_model.intercept_}")
# Regularized Ridge Regression
ridge_model = Ridge(alpha = 0.3)
ridge_model.fit(x_train, y_train)
print('Ridge Model coef: {}'.format(ridge_model.coef_))
# Regularized Lasso Regression
lasso_model = Lasso(alpha = 0.1)
lasso_model.fit(x_train, y_train)
print('Lasso model coef:{}'.format(lasso_model.coef_))
# Score Comparison
#Simple Linear Model
print(regression_model.score(x_train, y_train))
print(regression_model.score(x_test, y_test))
# Ridge regrssion
print(ridge_model.score(x_train, y_train))
print(ridge_model.score(x_test, y_test))
# Lasso regrssion
print(lasso_model.score(x_train, y_train))
print(lasso_model.score(x_test, y_test))
Source Console Object
Usage
Help Variable Explorer Plots Files
Console 1/A
...: ridge_model.fit(x_train, y_train)
Out[12]: Ridge(alpha=0.3)
In [13]: print('Ridge Model coef: {}'.format(ridge_model.coef_))
Ridge Model coef: [2.47057467 2.44494419 -1.78573889 -5.47285499 0.10115618 2.92319984
2.94492098 -0.57949986 0.34667456 0.37344909]
In [14]:
...: lasso_model = Lasso(alpha = 0.1)
...: lasso_model.fit(x_train, y_train)
...:
...: print('Lasso model coef:{}'.format(lasso_model.coef_))
Lasso model coef:[1.10693517 0. -0.71587138 -4.2127655 -0. 2.73245903
1.66333749 -0.63587683 0. 0.]
In [15]: print(regression_model.score(x_train, y_train))
...: print(regression_model.score(x_test, y_test))
...:
...: # Ridge regrssion
...: print(ridge_model.score(x_train, y_train))
...: print(ridge_model.score(x_test, y_test))
...:
...: # Lasso regrssion
...: print(lasso_model.score(x_train, y_train))
...: print(lasso_model.score(x_test, y_test))
0.8343770256960538
0.8513421387780066
0.8343617931312616
0.8518882171608505
0.8211445134781439
0.8577234201035425
IPython Console History
conda (Python 3.12.7) Completions: conda LSP: Python Line 54, Col 34 ASCII CRLF RW Mem 80%